

서울시 부동산 실거래가가 예측

목차

- 01 주제 소개 및 선정 이유
- 02 데이터 소개
- 03 데이터 공통 전처리
- 04 시각화 구체화

- 05 추천 시스템
- 06 시계열 전처리
- 07 시계열 모델 예측
- 08 결론 및 활용방안

주제 소개 및 선정 이유

Problem

어디서 살아야 하지?

- ✓ 대학 진학, 첫 자취, 학업·경험 중심의 이동 등 20대는 주거 수요가 매우 활발한 시기
- ✓ 경험 부족과 시장 정보의 비대칭성으로 인해 적정 월세가를 판단하기 어려움

Pain Point

기준 없는 월세 시장과 정보 불균형

- ✓ 보증금, 전용면적, 보행 거리 (역세권), 건축연한 등 다양한 변수가 혼재
- ✓ 청년층 대상의 허위/부풀려진 매물 리스크 및 주거비 부담 증가

Goal

서울시 부동산 월세 실거래가
시계열 예측 및 추천 대시보드 제작

서울시 부동산 전월세가 정보

- ✓ 출처: 서울 열린데이터 광장
- ✓ 내용:
자치구, 법정동, 건물명, 임대건물명, 전월세구분, 보증금, 임대료, 계약년도 등의 전월세 정보를 담고 있음
- ✓ 사용 데이터 기간:
시각화 및 추천시스템: 2021~2025년
시계열예측: 2016~2025년

공공데이터

주택/건설

활용사례 등록

URL 복사

목록 이동

서울시 부동산 전월세가 정보

서울 부동산 정보광장에서 제공하는 전월세가 정보입니다. 자치구, 법정동, 건물명, 임대건물명, 전월세 구분, 보증금, 임대료, 계약년도 등의 전월세 정보를 제공합니다.
※ 미리보기에서 필드명 접수년도로 필터선택 후 연도별로 조회 및 내려받기 하여주시기 바랍니다.
※ FILE탭의 파일을 다운로드하시면, 더욱 빠르게 다운로드하실 수 있습니다.

[전체 설명보기](#)

접수연도	자치구코드	자치구명	법정동코드	법정동명	지번구분	지번구
6	11560	영등포구	13200	신길동	1	대지
6	11170	용산구	10200	용산동2가	1	대지
6	11710	송파구	10600	삼전동	1	대지
6	11710	송파구	10800	문정동	1	대지
6	11530	구로구	10200	구로동	1	대지
6	11530	구로구	10200	구로동	1	대지
6	11320	도봉구	10500	쌍문동	1	대지
6	11650	서초구	10100	방배동	1	대지
6	11620	관악구	10100	봉천동	1	대지
6	11560	영등포구	11000	여의도동	1	대지
6	11380	은평구	10100	수색동	1	대지
6	11380	은평구	10100	수색동	1	대지
6	11320	도봉구	10700	창동	1	대지
6	11740	강동구	10200	고덕동	1	대지
6	11740	강동구	10200	고덕동	1	대지
6	11140	종구	17400	만리동2가	1	대지
6	11500	강서구	10800	공항동	1	대지
6	11140	종구	16200	신림동	1	대지

데이터 공통 전처리 파이프라인 개요

01

02

03

04

05

월세 추출 & 정규화

월세 데이터 추출 및
수치·날짜형 포맷 표준화

텍스트 데이터 파싱

계약기간 텍스트 파싱

결측치 처리

면적 대상 추출 및
필수·제도 변수 결측 정제

이상치 제거

극단치 및 오기입
수치 데이터 제거

파생변수 생성

평당 단가, 갱신 증감 등
분석·모델링 변수 생성

월세 추출 & 정규화

모든 연도 파일 통합

- **제도 도입 칼럼 정제**: 2021년부터 집계 시작된 5개 칼럼(계약기간, 신규계약구분, 갱신청구권사용 등)의 공백 정제 및 결측치('미제공') 표준화
- **수치형·이진 변수 변환**: 계약기간을 개월 수로 계산하고, 신규 계약 여부 및 갱신청구권 사용여부를 수치형으로 인코딩
- **결측 플래그(Flag) 생성**: 과거 연도 미집계 및 데이터 누락에 따른 편향을 방지하고자 항목별 결측 여부 변수(1/0) 추가 생성

기본 수치형 변환

```
num_cols = [  
    '임대면적',  
    '보증금(만원)',  
    '임대료(만원)',  
    '층',  
    '건축년도',  
    '종전보증금',  
    '종전임대료',  
    '자치구코드',  
    '법정동코드',  
    '접수년도'  
]  
  
for col in num_cols:  
    if col in df_all.columns:  
        df_all[col] = pd.to_numeric(df_all[col], errors='coerce')
```

월세 데이터 추출

```
df_all['전월세구분'] = df_all['전월세구분'].astype(str).str.strip()  
monthly_df = df_all[df_all['전월세구분'] == '월세'].copy()
```

 54.2%

전체 데이터 : 3,238,096건

월세 추출 데이터: 1,754,985건

월세 추출 & 정규화

날짜 파싱 및 시계열 추출

```
monthly_df['계약일_str'] = (
    monthly_df['계약일']
    .astype('string')
    .str.replace('-', '', regex=False)
    .str.strip()
)

monthly_df['계약연도'] = pd.to_numeric(
    monthly_df['계약일_str'].str[:4],
    errors='coerce'
)

monthly_df['계약월'] = pd.to_numeric(
    monthly_df['계약일_str'].str[4:6],
    errors='coerce'
)

# 이상한 월은 NaN 처리
monthly_df.loc[~monthly_df['계약월'].between(1, 12), '계약월'] = np.nan

# 계약연도 0은 NaN 처리
monthly_df.loc[monthly_df['계약연도'] == 0, '계약연도'] = np.nan

monthly_df[['계약일', '계약연도', '계약월']].head()
```

- 계약일자 포맷 정규화: 문자열 내 하이픈(-) 제거 및 공백 정제를 통해 YYYYMMDD 형식으로 일관화
- 시계열 파생변수 추출: 정규화된 문자열을 슬라이싱하여 계약연도(YYYY) 및 계약월(MM) 수치형 칼럼 생성
- 예외값 및 유효성 검증: 범위를 벗어난 월 값(1~12월 외) 및 잘못 입력된 연도(0년 등)를 NaN으로 예외 처리하여 데이터 정밀도 제어

텍스트 데이터 파싱

계약 기간을 개월 수로 변환 예: '23.01~25.01' → 24

```
def calc_contract_months(x):
    if pd.isna(x):
        return np.nan

    x = str(x).strip()

    if x in ['', 'nan', 'NaN', '미제공', '0', 'Unknown']:
        return np.nan

    x = x.replace("[", "").replace("]", "")
    x = x.replace(" ", "")

    match = re.match(r'(\d{2})\.(?!\d{2})~(\d{2})\.(?!\d{2})', x)

    if not match:
        return np.nan

    start_y, start_m, end_y, end_m = map(int, match.groups())

    start_y += 2000
    end_y += 2000

    months = (end_y - start_y) * 12 + (end_m - start_m)

    if months <= 0:
        return np.nan

    return months
```

- 결측치 및 예외 처리: NaN, 공백, '미제공', '0', 'Unknown' 등의 유효하지 않은 데이터는 np.nan으로 반환
- 문자열 정규화: 따옴표 특수문자 교체 및 모든 공백 제거
- 정규표현식 추출: YY.MM~YY.MM 연도·월 패턴을 인식하여 시작/종료 연월 분리
- 연도 연산: 2자리 연도 표기를 2000년대(+2000)로 변환하여 연·월 기준 총 개월 수 계산
- 유효성 검증: 계산된 개월 수가 0 이하(종료일이 시작일보다 빠른 경우 등)이면 np.nan 반환

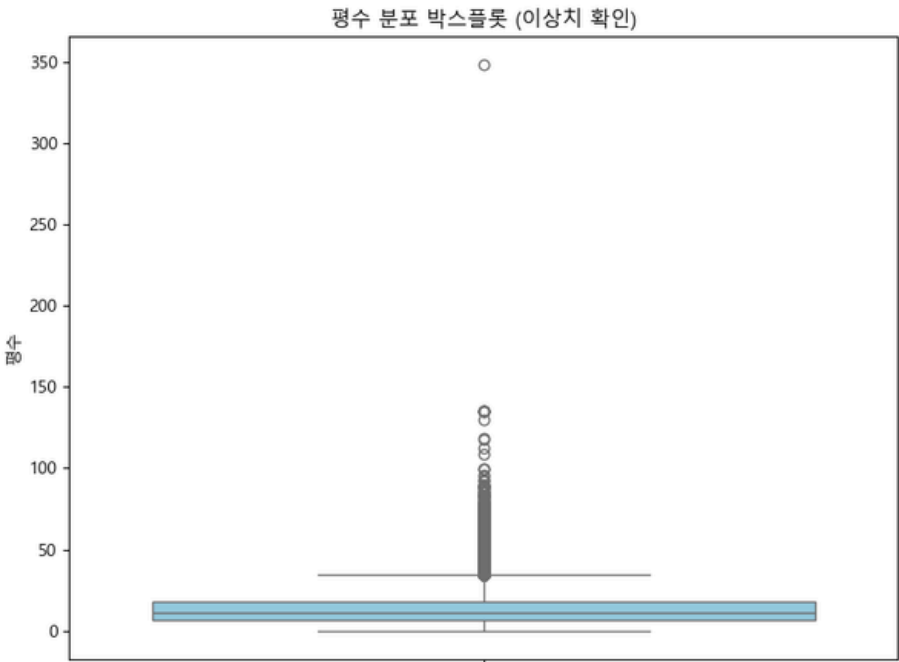
결측치 처리

면적 기준 필터링

2평 이상 & 50m² 미만으로 필터링, 결측 제거

```
df = monthly_df.copy()

df = df[
    (df['임대면적'].notna()) &
    (df['임대면적'] < 50) &
    (df['평수'] >= 2)
].copy()
```



최솟값	1사분위수 (Q1)	중앙값 (Q2)	3사분위수 (Q3)	최댓값
0.0	6.9	10.9	17.9	348.1

법정동명 / 법정동코드 / 자치구명 / 자치구코드 결측 제거

위치 정보는 분석 핵심 변수라 결측이면 제거

```
location_cols = ['법정동명', '법정동코드', '자치구명', '자치구코드']

existing_location_cols = [col for col in location_cols if col in df.columns]

df = df.dropna(subset=existing_location_cols).copy()
```

결측치 처리

제도 관련 변수 결측 처리

값을 대체하지 않고 NaN 그대로 유지

```
# 계약기간_개월
if '계약기간_개월' in df.columns:
    df['계약기간_결측여부'] = df['계약기간_개월'].isna().astype(int)

# 신규계약여부
if '신규계약여부' in df.columns:
    df['신규계약여부_결측여부'] = df['신규계약여부'].isna().astype(int)

# 갱신청구권사용여부
if '갱신청구권사용여부' in df.columns:
    df['갱신청구권_결측여부'] = df['갱신청구권사용여부'].isna().astype(int)

# 종전보증금, 종전임대료
for col in ['종전보증금', '종전임대료']:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')
        df[f'{col}_결측여부'] = df[col].isna().astype(int)
```

불필요한 칼럼 제거

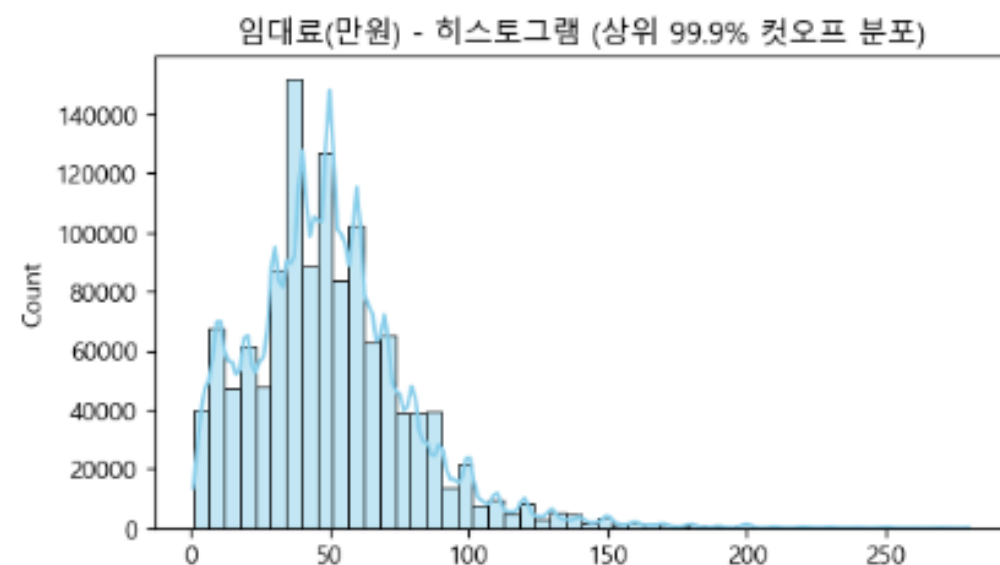
```
drop_cols = [
    '분번',
    '부번',
    '건물명',
    '지번구분',
    '지번구분코드',
    '계약일_str'
]

df = df.drop(columns=[col for col in drop_cols if col in df.columns])
```

이상치 제거

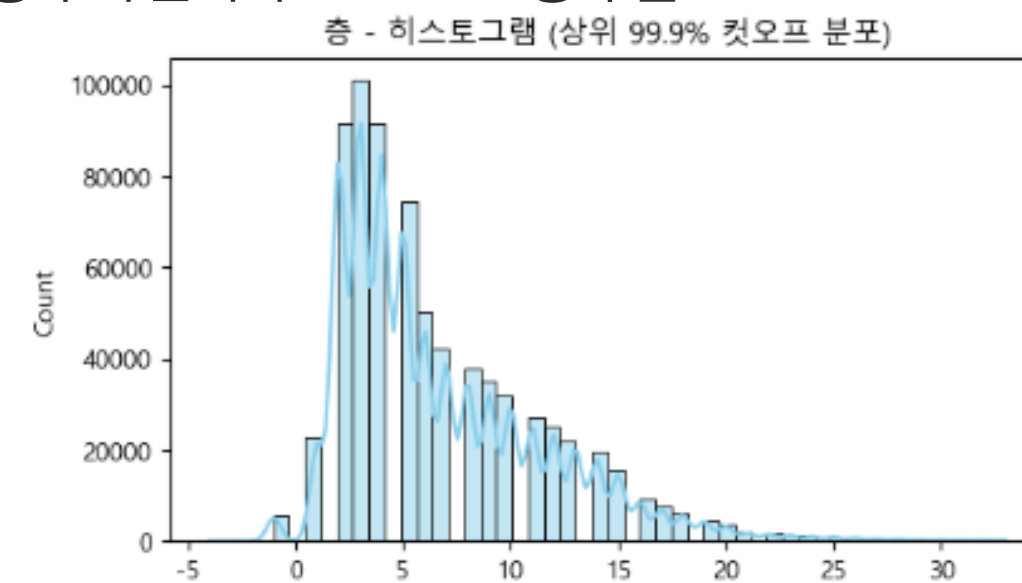
임대료

600만원 이하 추출



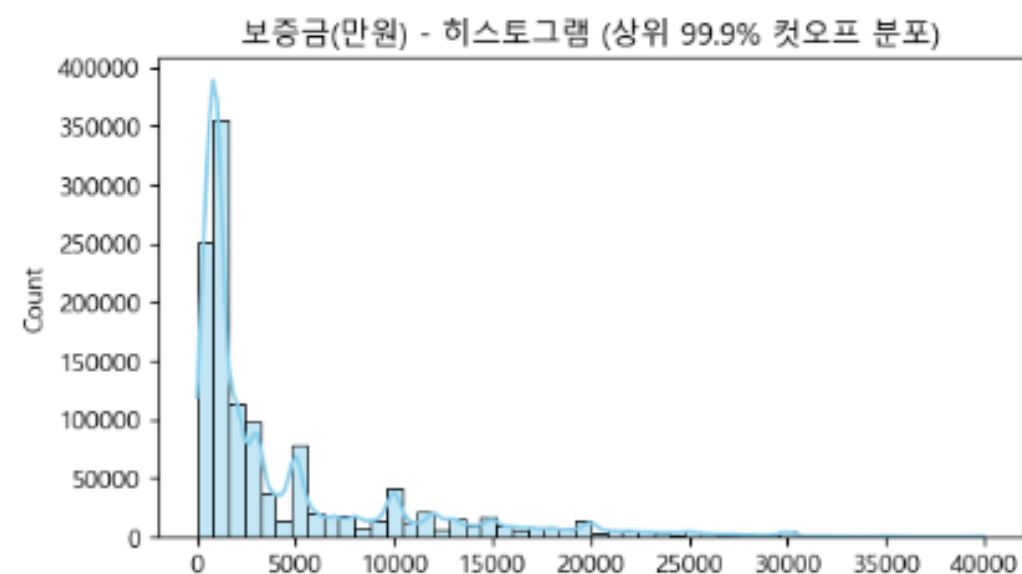
층

50m² 미만 소형 주거 분석이므로 -2~69층 추출



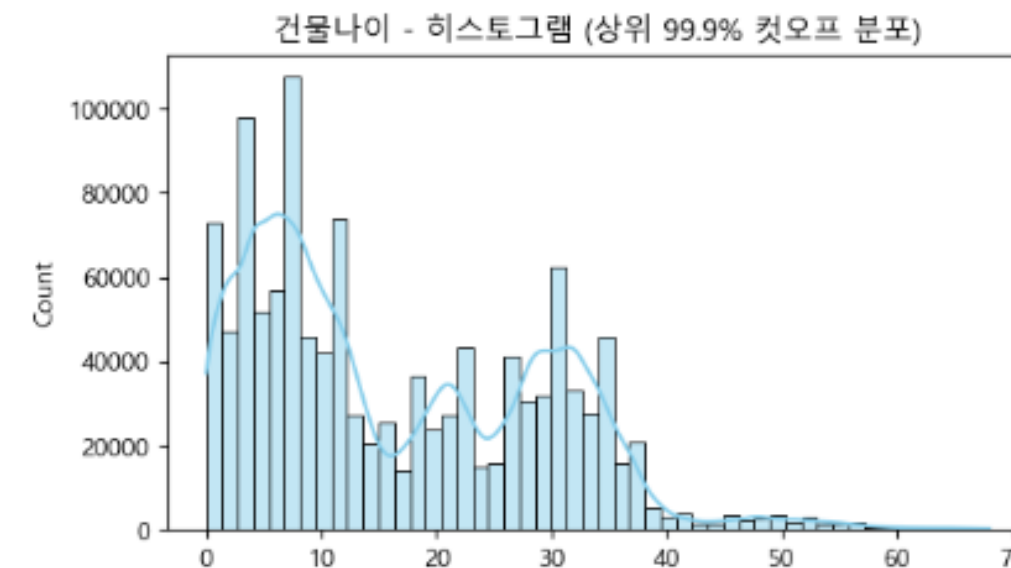
보증금

0 초과 100000만원 미만 추출



건물나이

0 이상 96년 이하 추출



이상치 제거

건축년도

너무 오래되거나 미래인 건축년도 제거

- 1930년 이상
- 건축년도 <= 계약연도

```
df = df[
    (df['건축년도'].isna()) |
    (
        (df['건축년도'] >= 1930) &
        (df['건축년도'] <= df['계약연도'])
    )
].copy()
```

계약기간

NaN은 유지, 값이 있는 경우에만 1~60개월 조건 적용

```
if '계약기간_개월' in df.columns:
    df = df[
        (df['계약기간_개월'].isna()) |
        (
            (df['계약기간_개월'] > 0) &
            (df['계약기간_개월'] <= 60)
        )
    ].copy()
```

종전보증금, 종전임대료

NaN은 유지, 값이 있는 경우에만 0 이상 조건 적용

```
for col in ['종전보증금', '종전임대료']:
    if col in df.columns:
        df = df[
            (df[col].isna()) |
            (df[col] >= 0)
        ].copy()
```

임대면적, 평수, 평당가격 생성

```
# 0 이하 값은 계산 불가능하므로 NaN 처리
monthly_df.loc[monthly_df['임대면적'] <= 0, '임대면적'] = np.nan
monthly_df.loc[monthly_df['보증금(만원)'] <= 0, '보증금(만원)'] = np.nan
monthly_df.loc[monthly_df['임대료(만원)'] <= 0, '임대료(만원)'] = np.nan

# 평수 생성
monthly_df['평수'] = monthly_df['임대면적'] / 3.3

# 평당가격: 월세 / 평수
monthly_df['평당가격'] = monthly_df['임대료(만원)'] / monthly_df['평수']

# 평당 보증금
monthly_df['평당_보증금'] = monthly_df['보증금(만원)'] / monthly_df['평수']
```

- [0 이하 예외 처리 \(NaN 처리\)](#): 계산 불가능한 **비정상 값**(임대면적, 보증금, 임대료 ≤ 0)을 NaN(결측치)으로 변환
- [평수](#): 임대면적(m^2) / 3.3 을 계산하여 평 단위로 규격화
- [평당 가격](#): 임대료(만원) / 평수 를 계산하여 면적 대비 월세 단가 산출
- [평당 보증금](#): 보증금(만원) / 평수 를 계산하여 면적 대비 보증금 단가 산출

파생변수 생성

건물 나이 생성

```
# 0은 잘못된 값으로 보고 NaN 처리
df.loc[df['건축년도'] == 0, '건축년도'] = np.nan

# 건물나이 생성
df['건물나이'] = df['계약연도'] - df['건축년도']

# 건축연도가 계약연도보다 큰 경우 NaN 처리
df.loc[df['건물나이'] < 0, '건물나이'] = np.nan
```

예시

계약연도	건축년도	건물나이
2021	1990.0	31.0
2020	1998.0	22.0
2021	2014.0	7.0

지하여부 생성

```
def check_basement(x):
    if pd.isna(x):
        return np.nan

    x_str = str(x).strip()

    if '지하' in x_str:
        return 1

    try:
        if float(x_str) <= 0:
            return 1
    except:
        return np.nan

    return 0

# 지하여부 먼저 생성
df['지하여부'] = df['층'].apply(check_basement)

# 이후 층 숫자형 변환
df['층'] = pd.to_numeric(df['층'], errors='coerce')
```

추가 파생변수 생성

```
# 성수기 여부: 1, 2, 7, 8월
df['is_성수기'] = np.where(
    df['계약월'].isna(),
    np.nan,
    df['계약월'].isin([1, 2, 7, 8]).astype(int)
)

# 법정동별 월세 중앙값
dong_rent_median = df.groupby('법정동코드')['임대료(만원)'].median()
df['법정동_월세_중앙값'] = df['법정동코드'].map(dong_rent_median)

# 법정동별 평당가격 중앙값
dong_price_median = df.groupby('법정동코드')['평당가격'].median()
df['법정동_평당가격_중앙값'] = df['법정동코드'].map(dong_price_median)

# 자치구별 월세 중앙값
gu_rent_median = df.groupby('자치구코드')['임대료(만원)'].median()
df['자치구_월세_중앙값'] = df['자치구코드'].map(gu_rent_median)

# 용도별 평균 평수 대비 크기
if '건물용도' in df.columns:
    type_area_median = df.groupby('건물용도')['평수'].transform('median')
    df['용도별_평균대비_평수비율'] = df['평수'] / (type_area_median + 0.001)

# 보증금 대비 월세 비율
df['보증금대비월세비율'] = np.where(
    df['임대료(만원)'].isna() |
    df['보증금(만원)'].isna() |
    (df['보증금(만원)'] <= 0),
    np.nan,
    df['임대료(만원)'] / df['보증금(만원)']
)
```

- 성수기 여부: 계약월이 이사 수요가 많은 1, 2, 7, 8월인지 여부를 1과 0으로 수치화
- 지역별 임대료 및 단가 중앙값: 법정동/자치구 단위의 월세 및 평당가격 중앙값을 매핑하여 지역별 시세 지표 생성
- 건물용도별 상대적 크기 비율: 같은 건물용도(아파트, 오피스텔 등)의 평수 중앙값 대비 해당 건물의 크기 비율 산출
- 보증금 대비 월세 비율: 보증금 금액에 따른 월세의 비중을 계산하여 거래 조건의 특성 반영

파생변수 생성

갱신계약 증감 변수 생성

보증금증감액 = 현재보증금 - 종전보증금

임대료증감액 = 현재임대료 - 종전임대료

증감률(%) = (증감액 / 종전금액) * 100

- 대상 조건: 종전 보증금 & 임대료가 존재하는 갱신 계약
- 생성 피처: 보증금증감액, 임대료증감액, 증감률(%)
- 결측치 처리: 신규 계약은 초기값 NaN 유지를 통한 구분

변화 유형 변수 생성

조건 (증감액 기준)	변동 유형 라벨
pd.isna(x) (결측)	해당없음
x > 0 (양수)	상승
x < 0 (음수)	하락
x == 0	동결

- change_type() 함수를 통한 변동 유형 라벨링
- 생성 피처: 보증금변화, 임대료변화



서울에서 자취방 구하기 솔루션

데이터 기반 서울시 전월세 분석 & 예측 솔루션 🚀



시장 현황 분석

지금 동네 시세는 얼마일까?



맞춤형 매물 추천

나만의 가성비 동네 찾기



미래 시세 예측

1년 뒤 내 방 월세는 오를까?



대시보드 접속하기



<https://seoultotaldashboard-aefmgfghsvxbocghjvfnzz.streamlit.app/>

home

 현황분석 대시보드

 맞춤형추천 대시보드

 미래예측 대시보드



서울에서 내게 딱 맞는 자취방 구하기

데이터 기반 서울시 전월세 분석 & 예측 솔루션 🚀

원하는 정보가 무엇인가요? 아래의 카드를 클릭하여 원하시는 대시보드로 바로 이동해 보세요.

 1. 시장 현황 분석

지금 동네 시세는 얼마일까?

5년 치 서울시 전월세 실거래가 데이터를 바탕으로 자치구와 법정동별 현재 시세, 보증금/월세 분포, 건물 특성을 한눈에 파악하고 비교해 보세요.

 현황 분석 보러가기

 2. 맞춤형 매물 추천

나만의 가성비 동네 찾기

어디서 살아야 할지 막막하신가요? 내 예산(보증금, 월세)과 원하는 방 크기를 입력하면, AI 기반 추천 알고리즘이 조건에 딱 맞는 최적의 동네를 추천해 드립니다.

 맞춤형 추천 받기

 3. 미래 시세 예측

1년 뒤 내 방 월세는 오를까?

지금 알아본 방, 1년 뒤엔 시세가 어떻게 될까요? 시계열 딥러닝 예측 모델이 그린 동네별 예상 평당가 지도를 통해 미래 시세를 예측하고 가성비를 판단하세요.

 미래 시세 확인하기

대시보드

home

현황분석 대시보드

맞춤형추천 대시보드

미래예측 대시보드

지역 선택

자치구 선택

전체

법정동 선택

전체

서울시 자치구/법정동별 월세 및 건물 현황

선택한 지역의 월세 기준 평당 가격 중앙값, 보증금, 건물 나이, 지하 여부 및 가격 범위를 분석합니다.

서울시 전체 월세 분석 결과 (1,242,901 건)

평당 가격 (중앙값)
6.2 만원

보증금 (중앙값)
2,000 만원

평균 건물 나이
16.6 년

지하/반지하 비중
0.5%
 총 5748 건

가격 범위 (최저 ~ 최대)

보증금 범위
최저 1 만원 ~ 최고 90,000 만원

월세(임대료) 범위
최저 1 만원 ~ 최고 600 만원

평당가격 범위
최저 0.1 만원 ~ 최고 101.1 만원

서울시 전체는 데이터가 많아 그래프는 10,000건을 무작위 샘플링하여 표시합니다.

보증금 vs 월세 분포

건물 용도별 분포

선택 지역 원본 상세 데이터 보기

추천시스템 입력 데이터

사용 컬럼

- 보증금
- 평당가격
- 평수
- 건물나이
- 자치구명·법정동명
- 계약연도
- 지하여부
- 건물용도



대시보드 선택 변수

필수

- 최소 보증금 (만원)
- 최대 보증금 (만원)
- 최소 평당 임대료 (만원/평)
- 최대 평당 임대료 (만원/평)
- 최소 평수
- 최대 평수
- 지역별 최소 거래 수
- 추천 지역 수

선택

- 건물 용도 선택
- 지하 매물 제외

추천시스템 입력 데이터

필수

```
# 입력값 검증
if deposit_min > deposit_max:
    raise ValueError(
        "최소 보증금은 최대 보증금보다 작아야 합니다."
    )

if price_per_pyeong_min > price_per_pyeong_max:
    raise ValueError(
        "최소 평당가격은 최대 평당가격보다 작아야 합니다."
    )

if area_min > area_max:
    raise ValueError(
        "최소 평수는 최대 평수보다 작아야 합니다."
    )

filtered = data[
    data['보증금(만원)'].between(
        deposit_min,
        deposit_max
    ) &
    data['평당가격'].between(
        price_per_pyeong_min,
        price_per_pyeong_max
    ) &
    data['평수'].between(
        area_min,
        area_max
    )
]
```

조건 충족 거래

= 보증금 범위 충족 $\cap$ 평당가격 범위 충족
 $\cap$ 평수 범위 충족 $\cap$ 선택 주거 조건 충족

선택

```
# 지하 제외
if (
    exclude_basement and
    '지하여부' in filtered.columns
):
    # 지하여부 결측치는 지하라고 단정하지 않고 유지
    filtered = filtered[
        filtered['지하여부'].isna() |
        (filtered['지하여부'] == 0)
    ].copy()

# 건물용도 선택
if (
    building_types is not None and
    len(building_types) > 0 and
    '건물용도' in filtered.columns
):
    filtered = filtered[
        filtered['건물용도'].isin(building_types)
    ].copy()
```

- 지하여부와 건물용도를 선택하면,
 각각의 조건도 충족하는 거래를 제시함
- 지하 여부 결측치는 지하라고 단정하지 않음

법정동별 거래 집계

집계 변수

- ✓ 조건 충족 거래 수
- ✓ 보증금 중앙값
- ✓ 평당가격 중앙값
- ✓ 월세 중앙값
- ✓ 평수 중앙값
- ✓ 건물나이 중앙값
- ✓ 거래가 존재한 연도 수
- ✓ 최근 거래 연도
- ✓ 가격 변동계수

✓ 조건적합률

$$\frac{\text{해당 지역의 조건 충족 거래수}}{\text{해당 지역의 전체 거래수}}$$

→ 거래 수만 사용하면 원래 거래량이 많은 지역이 항상 유리해질 수 있으므로, 지역 전체 거래 중 사용자 조건에 맞는 거래가 얼마나 되는지도 함께 반영함

```
# 전체 데이터의 지역별 거래 수
total_counts = (
    full_data
    .groupby(
        ['자치구명', '법정동명']
    )
    .size()
    .reset_index(name='지역전체_거래수')
)

result = region_summary.merge(
    total_counts,
    on=['자치구명', '법정동명'],
    how='left'
)

result['조건적합률'] = (
    result['조건충족_거래수'] /
    result['지역전체_거래수']
)
```

추천 점수 계산

점수 정규화

[역방향 점수 적용]

- 보증금
- 평당가격
- 건물나이
- 가격 변동계수

[정방향 점수 적용]

- 거래 수
- 평수
- 조건적합률
- 거래 지속성

[거래 안정성 통합]

: 거래량, 거래 지속성, 가격안정성을 혼합하여 거래 안정성 점수를 생성함

$$\begin{aligned} \text{거래안정성점수} &= 0.50 \times \text{거래량점수} \\ &+ 0.30 \times \text{거래지속성점수} \\ &+ 0.20 \times \text{가격안정성점수} \end{aligned}$$



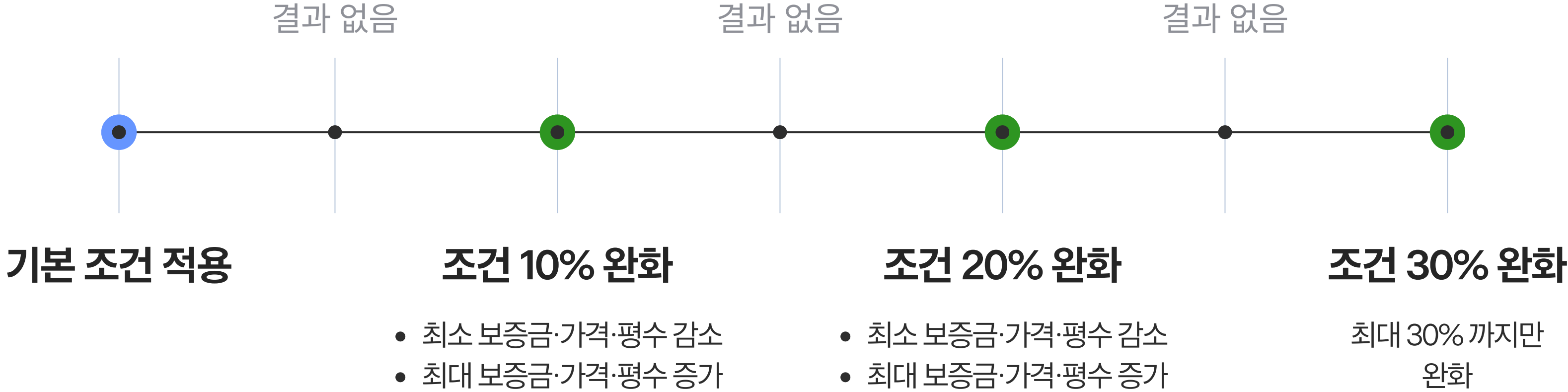
최종 추천점수 산정

$$\begin{aligned} \text{추천점수} = 100 \times & (0.20 \times \text{보증금점수} \\ & + 0.20 \times \text{평당가격점수} \\ & + 0.30 \times \text{거래안정성점수} \\ & + 0.15 \times \text{평수점수} \\ & + 0.10 \times \text{건물상태점수} \\ & + 0.05 \times \text{조건적합률점수}) \end{aligned}$$

*조건 충족 거래가 3건 미만인 지역은 제외

조건 완화 및 전체 추천 과정

- ✓ 조건을 만족하는 지역이 없는 경우, 사용자의 원래 조건을 최대한 유지하면서 추천 가능 범위를 단계적으로 확대함



추천 이유 포함

✔ 각 지역이 추천된 이유를 문장으로 함께 제공하여 추천 결과의 해석 가능성을 높임

점수	추천 이유 문장
보증금 점수 >= 0.7	보증금 부담이 비교적 낮음
평당가격 점수 >= 0.7	평당 임대료가 비교적 낮음
거래 안정성 점수 >= 0.7	조건에 맞는 거래가 안정적으로 존재함
평수 점수 >= 0.7	같은 조건에서 주거 면적이 넓은 편임
건물상태 점수 >= 0.7	건물 연식이 비교적 최근임
조건적합률 점수 >= 0.7	지역 내 조건 적합 매물 비율이 높음

희망 조건 입력

최소 보증금(만원)

500

-

+

최대 보증금(만원)

5000

-

+

최소 평당 임대료(만원/평)

2.00

-

+

최대 평당 임대료(만원/평)

8.00

-

+

최소 평수

5.00

-

+

최대 평수

12.00

-

+

☒ 지하 매물 제외

지역별 최소 거래 수

3

추천 지역 수

10

Fork

💡 서울 월세 거주지역 맞춤형 추천

2023~2025년 서울 월세 실거래 데이터를 바탕으로 보증금, 평당 임대료, 평수를 고려하여 거주 가능한 법정동을 추천합니다.

입력 조건을 만족하는 지역을 찾았습니다.

📌 조건 충족 거래

293,690건

🏆 추천 지역

10곳

★ 최고 추천점수

72.8점

📄 상세 추천 지역 리스트

순위	추천지역	추천점수	조건충족_거래수	조건적합률(%)	보증금_중앙값	평당가격_중앙값	월세_중앙값	평수_중앙값	건물나이_중앙값	추천이유
1	강동구 강일동	72.8	771	9.82	1000	2.75	28	8.87	5	보증금 부담이 낮
2	도봉구 방학동	72.38	1745	35.91	500	4.5	40	9.73	26	보증금 부담이 낮
3	은평구 불광동	71.03	2156	34.97	1000	4.28	40	9.82	28	보증금 부담이 낮
4	성북구 길음동	70.81	282	18.63	1500	2.78	27	10	15	보증금 부담이 낮
5	양천구 신월동	70.17	2123	30.83	1000	4.38	40	9.75	31	보증금 부담이 낮
6	송파구 마천동	69.55	1051	29.5	1000	3.87	35	9.33	32	보증금 부담이 낮
7	중랑구 면목동	69.32	6120	39.38	1000	4.95	40	9.09	30	보증금 부담



시계열 전처리

```
df['계약일'] = pd.to_datetime(df['계약연도'].astype(int).astype(str) + '-'  
+ df['계약월'].astype(int).astype(str).str.zfill(2) + '-01')  
  
df = df.sort_values('계약일').reset_index(drop=True)
```

계약연도/계약월 → 계약일(datetime) 생성, 정렬

```
dong_info = (  
    df.groupby('법정동코드')[['자치구코드', '자치구명', '법정동명']]  
    .agg(lambda x: x.mode().iloc[0]) # 동일 법정동코드 내 표기 흔들림 방지용 대표값(최빈값)  
    .reset_index()  
)  
  
print("고유 법정동코드 개수:", dong_info.shape[0])  
dong_info.head()
```

고유 법정동코드 개수: 85

법정동코드 ↔ 자치구명/법정동명 매핑 (동명이인 방지용)

법정동명만으로 그룹핑하면 서로 다른 자치구에 있는 동일 이름의동이 섞일 수 있다.
그룹 키는 반드시 고유한 법정동코드를 사용하고, 자치구명/법정동명은 표시용으로만 매핑.

```
# 전체 기간 누적 최소 거래건수  
MIN_TOTAL_TXN = 60  
# 전체 관측 개월 수 대비, 거래가 1건 이상 있었던 월의 비율(커버리지)  
MIN_COVERAGE = 0.5
```

전체 동 수: 85개
필터링 후 동 수: 62개 (누적거래 60건 이상 & 커버리지 50% 이상)

거래량 적은 동 필터링 (MIN_TOTAL_TXN, MIN_COVERAGE 기준)

구 단위(25개)에서 동 단위(약 400여 개)로 내려가면 동마다 월별 거래 건수가 크게 줄어든다.
거래가 거의 없는 동은 (1) 월별 평균값 자체가 노이즈에 추락하고, (2) 결측월이 많아 12개월 lag/rolling을 사실상 만들 수 없으므로 사전에 제외.

시계열 전처리

법정동 x 연 x 월 집계

ts_df 생성

```
ts_df = (
    df_dong.groupby(['법정동코드', '계약연도', '계약월'], as_index=False).agg(
        월세=('임대료(만원)', 'mean'),
        보증금=('보증금(만원)', 'mean'),
        평균평수=('평수', 'mean'),
        평균평당가격=('평당가격', 'mean'),
        거래건수=('임대면적', 'count'))

ts_df = ts_df.merge(dong_info, on='법정동코드', how='left')
```

결측월까지 포함한 완전한 월별 패널로 재구성

groupby + shift로 lag/rolling을 만들 때, 특정 동에 거래가 없는 달이 하나라도 있으면 shift(1)이 실제로는 '한 달 전'이 아니라 '그 이전 마지막 거래월'을 가리키게 되어 lag/rolling/target의 의미가 왜곡된다. 이를 막기 위해 동마다 관측 구간 전체를 빠짐없이 한 행씩 갖도록(거래가 없던 달은 NaN 행으로) 패널을 먼저 채운다.

계절성, lag, rolling, diff, month encoding, trend, target 생성

구 단위(25개)에서 동 단위(약 400여 개)로 내려가면 동마다 월별 거래 건수가 크게 줄어든다. 거래가 거의 없는 동은 (1) 월별 평균값 자체가 노이즈에 취약하고, (2) 결측월이 많아 12개월 lag/rolling을 사실상 만들 수 없으므로 사전에 제외.

```
# 거래건수는 없으면 실제로 '0건'이 맞으므로 0으로 채움. 가격류(월세/보증금/평수/평당가격)는
# 그 달에 정의 자체가 안 되는 값이므로 NaN을 그대로 유지한다.
ts_df['거래건수'] = ts_df['거래건수'].fillna(0)
ts_df = ts_df.merge(dong_info, on='법정동코드', how='left')
ts_df['계약일'] = pd.to_datetime(
    ts_df['계약연도'].astype(int).astype(str) + '-'
    + ts_df['계약월'].astype(int).astype(str).zfill(2) + '-01')
ts_df = ts_df.sort_values(['법정동코드', '계약일']).reset_index(drop=True)
```

패널 전체 행 수 (동 개수 x 개월 수): 11222
월세 결측 비율 (실제 거래가 없었던 달): 0.176

```
ts_df['is_성수기'] = ts_df['계약월'].isin([1, 2, 7, 8]).astype(int)
```

```
lag_list = [1, 2, 3, 6, 12]
for lag in lag_list:
    ts_df[f'월세_lag{lag}'] = (ts_df.groupby('법정동코드')['월세'].shift(lag))
```

```
rolling_windows = [3, 6, 12]
for w in rolling_windows:
    ts_df[f'rolling_mean{w}'] = (ts_df.groupby('법정동코드')['월세'].transform(lambda x: x.shift(1).rolling(w).mean()))
    ts_df[f'rolling_std{w}'] = (ts_df.groupby('법정동코드')['월세'].transform(lambda x: x.shift(1).rolling(w).std()))
```

```
ts_df['diff1'] = ts_df.groupby('법정동코드')['월세'].diff(1)
ts_df['diff3'] = ts_df.groupby('법정동코드')['월세'].diff(3)
```

```
ts_df['month'] = ts_df['계약일'].dt.month
ts_df['month_sin'] = np.sin(2 * np.pi * ts_df['month'] / 12)
ts_df['month_cos'] = np.cos(2 * np.pi * ts_df['month'] / 12)
```

```
ts_df['time_index'] = ts_df.groupby('법정동코드').cumcount()
```

```
ts_df['target'] = ts_df.groupby('법정동코드')['월세'].shift(-1)
```

시계열 전처리

```
train_mask, valid_mask, test_mask = split_mask(ts_df)

train = ts_df[train_mask].copy()
valid = ts_df[valid_mask].copy()
test = ts_df[test_mask].copy()

print("train:", train.shape, " valid:", valid.shape, " test:", test.shape)

train: (5756, 33)  valid: (672, 33)  test: (634, 33)
```

Train/Valid/Test 분리

ts_df 생성

```
prophet_cols = ['법정동코드', '법정동명', '자치구명', '계약일', '월세']

prophet_train = train[prophet_cols].rename(columns={'계약일': 'ds', '월세': 'y'})
prophet_valid = valid[prophet_cols].rename(columns={'계약일': 'ds', '월세': 'y'})
prophet_test = test[prophet_cols].rename(columns={'계약일': 'ds', '월세': 'y'})
```

Prophet용 데이터

groupby + shift로 lag/rolling을 만들 때, 특정 동에 거래가 없는 달이 하나라도 있으면 shift(1)이 실제로는 '한 달 전'이 아니라 '그 이전 마지막 거래월'을 가리키게 되어 lag/rolling/target의 의미가 왜곡된다. 이를 막기 위해 동마다 관측 구간 전체를 빠짐없이 한 행씩 갖도록(거래가 없던 달은 NaN 행으로) 패널을 먼저 채운다.

시계열 모델 예측

- ✓ 전통적 모델: PROPHET, SARIMA
- ✓ 머신러닝 모델: LGBM, XGBOOST, CATBOOST
- ✓ 딥러닝 모델: LSTM

```
WINDOW = 24    # 과거 24개월을 보고  
HORIZON = 12   # 향후 12개월 예측
```

```
def build_windows(raw, target_col, cutoff, dong_stats, window=WINDOW, horizon=HORIZON):  
    X_seq, X_dong, Y = [], [], []  
  
    for dong_code, grp in raw.groupby('법정동코드'):  
        grp = grp[grp['계약일'] <= cutoff].sort_values('계약일')  
        series = grp.set_index('계약일')[target_col].asfreq('MS')  
  
        mean_, std_ = dong_stats[dong_code]  
        scaled = ((series - mean_) / std_).values  
  
        n = len(scaled)  
        for start in range(0, n - window - horizon + 1):  
            x = scaled[start:start + window]  
            y = scaled[start + window: start + window + horizon]  
  
            if np.isnan(x).any() or np.isnan(y).any():  
                continue  
  
            X_seq.append(x)  
            X_dong.append(dong_to_idx[dong_code])  
            Y.append(y)  
  
    return np.array(X_seq), np.array(X_dong), np.array(Y)
```

시계열 모델 예측

예측 결과

MODEL	MAPE	RMSE
PROPHET	13.26%	1.541
SARIMA	13.56%	1.849
LGBM	14.55%	1.499
XGBOOST	14.99%	1.563
CATBOOST	14.48%	1.492
LSTM	12.12%	1.399

Model Performance (MAPE)

LightGBM	XGBoost	Catboost	LSTM
14.55%	14.99%	14.48%	12.12%

머신러닝 모델링에 적용한 기법

- 교차검증 (TimeSeriesSplit)

```
from sklearn.model_selection import TimeSeriesSplit
```

```
# 시계열 5-Fold 교차 검증
tscv = TimeSeriesSplit(n_splits=5)
cv_scores = []

for train_idx, val_idx in tscv.split(X):
    X_tr, y_tr = X.iloc[train_idx], y.iloc[train_idx]
    X_va, y_va = X.iloc[val_idx], y.iloc[val_idx]

    model = xgb.XGBRegressor(**params)
    model.fit(
        X_tr, y_tr,
        eval_set=[(X_va, y_va)],
        verbose=False
    )
```

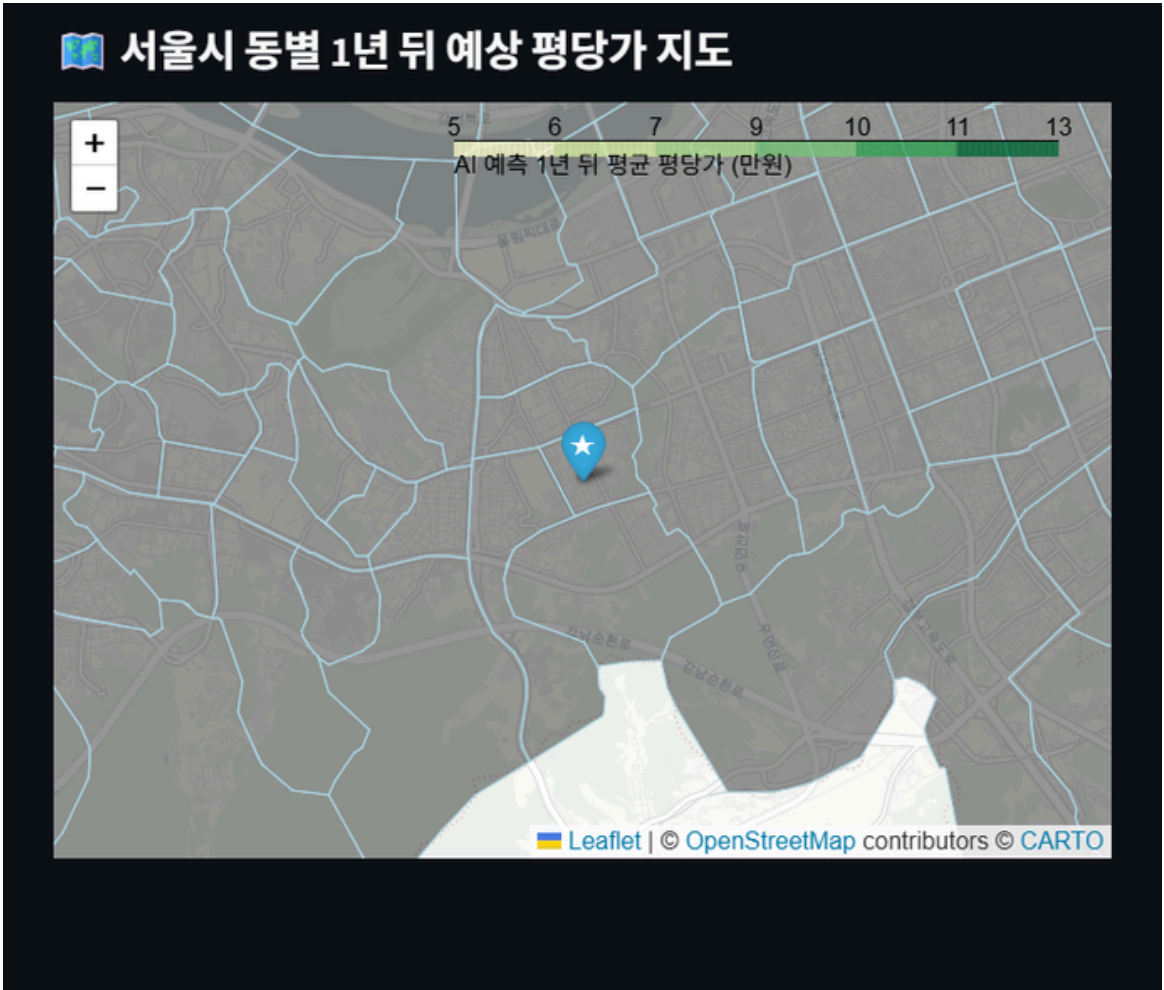
✓ 트리 기반 머신러닝의 한계

- 훈련 데이터의 최댓값과 최솟값 범위 내에서만 분할하는 특징
- 부동산 시장처럼 과거 데이터에서 더 상승하는 변화 예측에 구조적으로 어려움

✓ LSTM 선정

- 타임시리즈 데이터 내의 장기적인 Trend 패턴 인식에 유리
- 부동산 시세 상승폭을 유연하게 학습하여 높은 정확도 달성

시계열 예측 대시보드 코드



공간 데이터 매핑 (Choropleth)

단순 수치 제공을 넘어, [GeoJSON](#) 공간 데이터와 LSTM 예측 결과를 결합하여 직관적인 지도 구현)

```
folium.GeoJson(  
    seoul_geojson,  
    name="Click Layer",  
    style_function=lambda x: {'fillColor': 'transparent',  
                               'color': 'lightblue', 'weight': 1, 'fillOpacity': 0},  
    tooltip=folium.GeoJsonTooltip(fields=['adm_nm'], aliases=['동 이름:']),  
).add_to(m)
```

시계열 예측 대시보드 코드

✓ 사용자 설정 예산 기반 통계 정보 도출

- 해당 동네의 예측값을 자치구 그룹 평균과 비교연산
- 사용자 입력 예산 및 평수에 비해 1년 후 미래의 매물 시세 증감률 도출

```
diff_gu = predicted_pyeong_price - gu_avg_price
gu_diff_pct = (diff_gu / gu_avg_price) * 100

st.metric(label=f"AI 예측 {curr_dong} 1년 뒤 평당가", value=f"{predicted_pyeong_price:.1f}만원/평",
          delta=f"같은 구(주변) 1년 뒤 평균 대비 {abs(gu_diff_pct):.1f}% {'비쌈' if diff_gu > 0 else '저렴'}",
          delta_color="inverse" if diff_gu > 0 else "normal")
```

```
st.markdown(f"### 🏠 {int(area)}평 현재가 vs 1년 뒤 예측가 비교")
diff_rent = expected_rent - user_rent
rent_diff_pct = (diff_rent / user_rent) * 100 if user_rent > 0 else 0

st.metric(label="딥러닝 예상 1년 뒤 적정 월세", value=f"{expected_rent}만원")
```

🏠 방배동 1년 뒤 미래 시세 분석

🔥 1년 뒤 동네 상대 비교

AI 예측 방배동 1년 뒤 평당가

6.9만원/평

↑ 같은 구(주변) 1년 뒤 평균 대비 7.0% 저렴

💡 1년 뒤 방배동은 주변 동네들 평균(7.4만원)보다 저렴하여 가성비가 좋을 전망입니다.

🛏️ 10평 현재가 vs 1년 뒤 예측가 비교

딥러닝 예상 1년 뒤 적정 월세

68만원

✅ 지금 알아보신 60만원 방은 1년 뒤에 8만원 오를 전망입니다 (+13.3%)

시계열 예측 대시보드의 한계점과 프로젝트의 기대효과

한계점

- **거시경제 지표 미반영**
 - 기준 금리, 부동산 시장 정책 등과 같은 외부 환경 변수를 모두 고려하여 시계열 모델에 적용하지 않았다는 한계
- **비정형 데이터 반영 부재**
 - 채광, 내부 옵션 등 정성적 요인을 가격 예측에 반영 하지 못함
- **일부 법정동에 대한 예측 결과 부재**
 - 시계열 예측 특성상 거래 매물 수가 미래 예측하기에 부족한 경우, 해당 법정동의 예측이 어려움 → 모든 법정동을 예측하진 못함



기대 효과 및 활용방안

- **세입자의 정보 비대칭성 해소**
 - 객관적인 데이터와 예측가를 통해 2030 세대의 합리적 주거 선택 지원
- **서비스 고도화 적용**
 - 부동산 플랫폼 기업(다방, 직방 등)의 시세 리포트 및 AI 매물 추천 기능으로 확장 가능
- **지자체 주거 복지 정책 활용**
 - 시계열 예측 시스템으로 거래가 급증 지역을 미리 예측하여 지원금 정책 근거로 활용

경청해 주셔서 감사합니다!

OB 1조

김경민, 유재은, 전예원, 편한비
